ELEC60004/70068. MACHINE REASONING

Information for Students

KE Calculus

α-Formula Rules

$$\frac{p \wedge q}{\begin{array}{c} p \\ q \end{array}} \qquad \frac{\neg(p \vee q)}{\begin{array}{c} \neg p \\ \neg q \end{array}} \qquad \frac{\neg(p \rightarrow q)}{\begin{array}{c} p \\ \neg q \end{array}}$$

β-Formula Rules

$$\frac{p \rightarrow q}{\begin{array}{l} p \\ \quad q \end{array}} \qquad \frac{p \rightarrow q}{\begin{array}{l} \quad \neg q \\ \neg p \end{array}} \qquad \frac{p \vee q}{\begin{array}{l} \neg p \\ \quad q \end{array}} \qquad \frac{p \vee q}{\begin{array}{l} \quad \neg q \\ p \end{array}} \qquad \frac{\neg(p \wedge q)}{\begin{array}{l} p \\ \quad \neg q \end{array}} \qquad \frac{\neg(p \wedge q)}{\begin{array}{l} \quad q \\ \neg p \end{array}}$$

Double Negation Elimination

$$\frac{\neg(\neg p)}{p}$$

η-Formula Rules

$$\frac{p \leftrightarrow q}{\begin{array}{l} p \\ \quad q \end{array}} \qquad \frac{p \leftrightarrow q}{\begin{array}{l} \neg p \\ \quad \neg q \end{array}} \qquad \frac{p \leftrightarrow q}{\begin{array}{l} \quad q \\ p \end{array}} \qquad \frac{p \leftrightarrow q}{\begin{array}{l} \quad \neg q \\ \neg p \end{array}} \qquad \frac{\neg(p \leftrightarrow q)}{\begin{array}{l} p \\ \quad \neg q \end{array}} \qquad \frac{\neg(p \leftrightarrow q)}{\begin{array}{l} \quad q \\ \neg p \end{array}} \qquad \frac{\neg(p \leftrightarrow q)}{\begin{array}{l} \neg p \\ \quad q \end{array}} \qquad \frac{\neg(p \leftrightarrow q)}{\begin{array}{l} \quad \neg q \\ p \end{array}}$$

Branching Rule (PB)

$$\begin{array}{ccc} & \diagup\diagdown & \\ p & & \neg p \end{array}$$

Closure Rule

$$\frac{\begin{array}{c} p \\ \neg p \end{array}}{\times}$$

γ and δ Formula Rules (Predicate Logic)

γ rule	$\dfrac{\forall x.A}{A_{\{c/x\}}}$	$\dfrac{\neg\exists x.A}{\neg A_{\{c/x\}}}$	for any constant c
δ rule	$\dfrac{\exists x.A}{A_{\{c/x\}}}$	$\dfrac{\neg\forall x.A}{\neg A_{\{c/x\}}}$	for a constant c new to the branch

Modal Formula Rules (Modal Logics K and S5)

K

$\Box$-rule	$\dfrac{\sigma : \Box p}{\sigma n : p}$	$\dfrac{\sigma : \neg\Diamond p}{\sigma n : \neg p}$	for any available σn
$\Diamond$-rule	$\dfrac{\sigma : \Diamond p}{\sigma n : p}$	$\dfrac{\sigma : \neg\Box p}{\sigma n : \neg p}$	for σn unrestricted

S5

$\Box$-rule	$\dfrac{i : \Box p}{j : p}$	$\dfrac{i : \neg\Diamond p}{j : \neg p}$	for any available integer j
$\Diamond$-rule	$\dfrac{i : \Diamond p}{j : p}$	$\dfrac{i : \neg\Box p}{j : \neg p}$	for j unrestricted

1. This question is about search space formalisation and the General Graph Search Engine (GGSE).

a) In the context of problem-solving search using the GGSE, explain the difference between a *state* and a *node*, and define how a *state*, a *node*, a *path* and a *graph* can be represented in Prolog, so that it can be used with the GGSE.

[6]

b) The Blocks World scenario imagines a set of movable coloured blocks situated on a table, as illustrated in Figure 1.1.

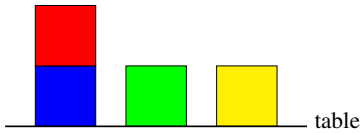


Figure 1.1 Blocks World: Initial State

There is a robot which can perform two actions: it can *stack* a block by taking any single block that is on the table and putting it on any other block, and it can *unstack* any block that is on another block. The goal is to build stacks of blocks called *towers* in a specified arrangement of colours.

Define a state space formulation for the Blocks World in Prolog, and give an example using Figure 1.1.

Explain why your formulation does (or does not) include explicit representation of the table.

[4]

c) Write state-change rules for the two robot actions, *stack* and *unstack*.

Explain why, although there could be multiple *stack* and *unstack* actions that the robot could perform in any given state, only two state-change rules are needed by the GGSE for this problem.

Explain clearly how unification and/or the conditions on the state-change rules are used to determine which of the two actions can be (validly) performed (i.e., how the conditions distinguish between a single block and a tower).

[10]

```
/*
** (a)
*/
State Space
-- state: data structure representing  state of the problem
-- node: data structure including state, but to the information as well,
e.g. f-cost, g-cost, etc.

Representation in Prolog:
-- state:compound term encapsulating state information (i.e. value of each unit
of information)
-- node:compound term with functor, or tuple, including state and possibly more
information, for example, f-, g- and h-cost, rule used to reach state, etc
-- path: list of nodes
-- graph: list of paths

/*
** (b)
*/
List of lists
[ [red,blue], [green], [yellow] ]

Formulation does not include explicit representation of table because
towers are represented from top to bottom with head of the list being
the top block, so the last element of a list is assumed implicitly to
be on the table.

Therefore the table does not include explicit representation of the table,
same as the robot, so this domain element is abstracted away, making it
faithful to the scenario fit is meant to represent but simpler to represent
and more efficient to reason with (in principle).

/*
** (c)
*/

state_change( stack, Towers, NewTowers ) :-
        append( Front, [ [Block] | Back ], Towers ),     %generate each single block
        append( Front, Back, Temp ),                     %all other towers
        append( F2, [ Tower | B2], Temp ),               %generate each other tower
        append( F2, [ [Block|Tower] | B2 ], NewTowers ). %stack the block on the tower

state_change( unstack, Towers, NewTowers ) :-
        append( Front, [ [Block|T] | Back ], Towers ),     %generate each tower
        T \= [],                                          %ensure not single
        append( Front, [ [Block], T | Back ], NewTowers ). %unstack block from tower

We are (ab)using the non-determinism of append in conjunction with the findall
of one-step-extensions
-- for the stack action, to backtrack and generate each singleton block on the table,
and each other object it could be stacked on
-- for the unstack action, to backtrack and generate each tower

-- for the stack action, the pattern [Block] only unfies with singleton blocks and
not towers, which are lists of two or more blocks
-- for the unstack action, the pattern [Block|T] could match with a singleton
block, e.g. [green], and with a tower, e.g. [red,b;ue]. However, the condition
T \= [] ensures that singleton blocks are not generated, since e.g. unifying [green]
with [Block|T] gives Block=green and T=[]
```

2. This question is about search space formalisation, the General Graph Search Engine (GGSE) and search algorithms.

a) Explain the difference between Uniform Cost, Best First and A^* search algorithms with respect to the order in which they test and expand paths.

Compare and contrast the three algorithms with respect to appropriate criteria.

[6]

b) Give the two conditions that a heuristic h must satisfy if it is to be used in the A^* search algorithm.

Explain why these two conditions must be satisfied.

Explain why these conditions might make one heuristic perform "better" than another in A^* search.

[6]

c) The Farmer-Wolf-Goat-Cabbage (FWGC) problem used a state space formulation that represented the domain objects according to their locations. The Castle Vanya problem used a state space formulation that represented the domain objects according to their crossing times, and the locations as two lists.

Reformulate the FWGC problem, but now representing each domain object as a term and distinguishing between left and right hand banks. Give the initial state, the goal state, and specify at most **two** state change rules.

[8]

(a)

uniform cost: selects node on search frontier with lowest g-cost (actual cost of path from start node to frontier node as computed by some path cost function g)

best first: selects node on search frontier with lowest h-cost (estimated cost of path from frontier node to goal node as computed by some heuristic function h)

A^*: selects node on search frontier with lowest f-cost (sum of g-cost and h-cost, giving lowest estimated cost of path from start node to goal node going through this frontier node)

criteria	uniform cost	best first	A^*
optimal	yes	no	yes
complete	yes	no	yes
time complexity	$O(b^d)$	$O(b^m)$	$O(b^d)$
space complexity	$O(b^d)$	$O(b^m)$	$O(b^d)$

where b is branching factor, d is depth of solution, m is maximum depth of tree

A^* is provably optimal for algorithms of this type

(b)

Condition 1: $h(G) = 0$ iff G is a goal state

Condition 2: $h(n)$ is admissible (never over-estimates the cost of getting from n to a goal state)

Requirement for Condition 1 actually follows from Condition 2: if the h-cost of getting to a goal node was not 0, then the heuristic would not be admissible.

- A^* always tests and expands one of the nodes with the lowest f-cost
  - If we estimate $h_o > h_{actual}$, so $g(n) + h_o > g(n') + h(n')$, then we might find a path from S to a sub-optimal goal G' through n'
  - If we estimate $h_u < h_{actual}$, at some point $g(n) + h_u < g(n') + h(n')$, so n will be expanded
  - If we always under-estimate, at some point we will expand the solution path ending in G before any solution path ending in G'
- Imagine drawing a histogram with increasing f-cost along the x-axis
  - Then map all the nodes onto the histogram
  - A* will expand all those nodes to the left of the f^* bar
  - Therefore the 'trick' is to get as many nodes to the right of the f^* bar ...
  - ... without ever over-estimating the h-cost
- Make $h(n)$ as large as possible (without over-estimating)
  - A* expands all nodes for which $f(n) < f^*$
  - So A* expands all nodes for which $h(n) < f^* - g(n)$
  - Therefore if $h_2(n) \geq h_1(n)$, for all n, h_2 may expand fewer nodes

```
/*
** (c)
*/
initial state: state( [f,w,g,c], [] )
goal state: state( [], [f,w,g,c] )

statechange( farmerL2R, ([f|Left],Right), (NextL,[f|NextR]) ) :- %keep f at front
        pickNoneOrOne( Left, NoneOrOne, NextL ),                %take 0 or 1 from left
        safe( NextL ),                                          %ensure NextL is safe
        append( NoneOrOne, Right, NextR ).                      %add to right

statechange( farmerR2L, (Left,[f|Right]), ([f|NextL],NextR) ) :- %keep f at front
        pickNoneOrOne( Right, NoneOrOne, NextR ),               %take 0 or 1 from right
        safe( NextR ),                                          %ensure NextR is safe
        append( NoneOrOne, Left, NextL ).                       %ad to left

pickNoneOrOne( Left, [], Left ).                %farmer goes on her own
pickNoneOrOne( Left, [One], Rest ) :-           %return list consistent with none case
        append( Front, [One|Back], Left ),      %generate one object to take
        append{ Front, Back, Rest ).            %leave the rest

safe( Bank ) :-
        member( w, Bank ),                      %if the wolf is alone on the bank...
        \+ member( g, Bank ).                   %...then the goat better not be
safe( Bank ) :-
        member( g, Bank ),                      %if the goat is alone on the back...
        \+ member( c, Bank ).                   %...then the cabbage better not be
```

3. This question is about knowledge representation and reasoning with logic and Prolog.

a) Explain what it means for a logical system (e.g. propositional logic, predicate logic) to be decidable.

Explain what it means for an algorithm for deciding the satisfiability of a logical formula to be *sound* and *complete*.

[6]

b) Explain how well-formed formulas of propositional logic can be represented by Prolog terms.

Specify an algorithm for a tautology checker which could be implemented in Prolog, whereby querying the checker succeeds if a given term is a tautology. (Only the steps in the algorithm are required, not the Prolog code.)

Explain the relationship between the tautology checker and truth tables. Indicate the limitations of this algorithm for tautology checking in practice.

[6]

c) Consider the following English statements:

All cats are smarter than all dogs.
All dogs are smarter than all muskrats.
All cool animals are cats.
All dumb animals are dogs.
'Smarter than' is a transitive relation.
Doogle is a cool animal.
Boris is a dumb animal.
Elon is a muskrat.

Express these statements in first-order predicate logic, and convert them to a set of Horn clauses.

[4]

d) Prove, using resolution and proof by negation, that *Doogle is smarter than Elon*. The proof should show all the unifiers.

[4]

(a) Decidable: a logical system or language is decidable if there is an effective method for determining whether arbitrary formulas are tautologies or theorems of the system/language, — or not, i.e. it must finish true or false

Soundness: if the algorithm proves something, then it is an entailment

Completeness, if there is an entailment, then the algorithm can prove it

(b)

Define the logical connectives as Prolog operators; then replace each connective in the formula (wff) by the corresponding operator, and replace each propositional symbol by the same Prolog variable

- compute all possible combinations of true/false assignment for each variable (use findall to generate all valuations)
- evaluate each combination to see if it returns true
- if every combination returns true, then the formula is a tautology

The first step effectively computes every row of the truth table.

The length of the list is exponential in the number of propositional symbols in the formula. Moreover, only works for propositional logic, can't do tautology checker in other logics using truth tables because the valuation function is different

(c)

$\forall x.\forall y.(cat(x) \wedge dog(y)) \rightarrow smrtr(x,y)$
$\forall x.\forall y.(dog(x) \wedge muskrat(y)) \rightarrow smrtr(x,y)$
$\forall x.cool(x) \rightarrow cat(x)$
$\forall x.dumb(x) \rightarrow dog(x)$
$\forall x.\forall y.\forall z.smrtr(x,y) \wedge smrtr(y,z) \rightarrow smrtr(x,z)$
$cool(Doogle)$
$dumb(Boris)$
$muskrat(Elon)$

1 $\neg cat(X1) \vee \neg dog(Y1) \vee smrtr(X1,Y1)$
2 $\neg dog(X2) \vee \neg muskrat(Y2) \vee smrtr(X2,Y2)$
3 $\neg cool(X3) \vee cat(X3)$
4 $\neg dumb(X4) \vee dog(X4)$
5 $\neg smrtr(X5,Y5) \vee \neg smrtr(Y5,Z5) \vee smrtr(X5,Z5)$
6 $cool(Doogle)$
7 $dumb(Boris)$
8 $muskrat(Elon)$

(d)

9	neg conc	$\neg smrtr(Doogle,Elon)$	
10	5,9	$\neg smrtr(Doogle,Y5) \vee \neg smrtr(Y5,Elon)$	$\{X5 \mapsto Doogle, Z5 \mapsto Elon\}$
11	1,10	$\neg cat(Doogle) \vee \neg dog(Y1) \vee \neg smrtr(Y1,Elon)$	$\{X1 \mapsto Doogle, Y1 \mapsto Y5\}$
12	3, 11	$\neg cool(Doogle) \vee \neg dog(Y1) \vee \neg smrtr(Y1,Elon)$	$\{X3 \mapsto Doogle\}$
13	6, 12	$\neg dog(Y1) \vee \neg smrtr(Y1,Elon)$	
14	4, 13	$\neg dumb(X4) \vee \neg smrtr(X4,Elon)$	$\{X4 \mapsto Y1\}$
15	7, 14	$\neg smrtr(Boris,Elon)$	$\{X4 \mapsto Boris\}$
16	2, 15	$\neg dog(Boris) \vee \neg muskrat(Elon)$	$\{X2 \mapsto Boris, Y2 \mapsto Elon\}$
17	4, 16	$\neg dumb(Boris) \vee \neg muskrat(Elon)$	$\{X4 \mapsto Boris\}$
18	7, 17	$\neg muskrat(Elon)$	
19	8, 18	$\bot$	

4. This question is about reasoning in Propositional Logic, Predicate Logic, and Propositional Modal Logic, using the **KE** calculus.

Important: any answer to any part that doesn't use line numbers and rule references to indicate how new formulas have been derived will not get full marks.

a) In the *Obligatio Game*, a finite number n of rounds is chosen, depending on the severity of the exam. The examiner then gives the candidate successive assertions, $\phi_1, \phi_2, \ldots, \phi_n$ that she or he has to either *accept* or *reject* as each one is put forward. In the former case, ϕ_i is added to the candidate's stock of commitments; in the latter case, the negation $\neg\phi_i$ is added.

The candidate passes the exam if she maintains the consistency of her stock of commitments throughout all n assertions.

Suppose the candidate is exposed (successively) to the following three ($n = 3$) assertions:

1. $p \vee q$
2. $\neg((p \wedge q) \rightarrow r)$
3. $(p \wedge q) \leftrightarrow r$

Using the **KE** calculus, show what sequences of answers would ensure that the student passes the exam.

(Hint: use **KE** for model building, for all combinations of *accept* or *reject* of the assertions. If the proof has at least one open branch, then this implies a consistent sequence of answers.)

[8]

b) Prove the following syllogism using the **KE** calculus for first-order Predicate Logic:

All martyrs are phlegmatic
Some coughers are martyrs
Therefore, some coughers are phlegmatic

[3]

c) Define a Kripke Model, and explain how it is used to give a semantics for formulas of Modal Logic.

[3]

d) Using the **KE** calculus for Propositional Modal Logic, prove that the characteristic formulas for serial, symmetry and transitivity relations are theorems of logic **S5**:

(*i*) $\Box P \rightarrow \Diamond P$
(*ii*) $P \rightarrow \Box\Diamond P$
(*iii*) $\Diamond P \rightarrow \Box\Diamond P$

[6]

```
/*
** (a)
*/

[p + q, - (p & q) -> r, p & q <-> r]
   1     p + q
   2     - (p & q) -> r
   3     p & q <-> r
   4     br-1  3              p & q
   5           eta   3,4      r
   6           alpha 4        p
   7           alpha 4        q
               open [p, q, r]
   8     br-2  3              - (p & q)
   9           eta   3,8      - r
  10           beta  2,8      r
               close 10,9

[p + q, - (p & q) -> r, - (p & q <-> r)]
   1     p + q
   2     - (p & q) -> r
   3     - (p & q <-> r)
   4     br-1  3              p & q
   5           eta   3,4      - r
   6           alpha 4        p
   7           alpha 4        q
   8           beta  2,5      - (- (p & q))
   9           dne   8        p & q
  10           alpha 9        p
  11           alpha 9        q
               open [p, q, - r]
  12     br-2  3              - (p & q)
  13           eta   3,12     r
  14           br-1  1              p
  15                 beta  12,14    - q
                     open [p, r, - q]
  16           br-2  1              - p
  17                 beta  1,16     q
                     open [q, r, - p]

[p + q, - (- (p & q) -> r), p & q <-> r]
   1     p + q
   2     - (- (p & q) -> r)
   3     p & q <-> r
   4     alpha 2        - (p & q)
   5     alpha 2        - r
   6     eta   3,4      - r
   7     br-1  4              - p
   8           beta  1,7      q
               open [q, - p, - r]
   9     br-2  4              p
  10           beta  4,9      - q
               open [p, - q, - r]

[p + q, - (- (p & q) -> r), - (p & q <-> r)]
   1     p + q
   2     - (- (p & q) -> r)
   3     - (p & q <-> r)
   4     alpha 2        - (p & q)
   5     alpha 2        - r
   6     eta   3,4      r
         close 6,5
```

```
[- (p + q), - (p & q) -> r, p & q <-> r]
   1     - (p + q)
   2     - (p & q) -> r
   3     p & q <-> r
   4     alpha 1          - p
   5     alpha 1          - q
   6     br-1  3               p & q
   7           eta   3,6       r
   8           alpha 6         p
   9           alpha 6         q
               close 8,4
  10     br-2  3               - (p & q)
  11           eta   3,10      - r
  12           beta  2,10      r
               close 12,11

[- (p + q), - (p & q) -> r, - (p & q <-> r)]
   1     - (p + q)
   2     - (p & q) -> r
   3     - (p & q <-> r)
   4     alpha 1          - p
   5     alpha 1          - q
   6     br-1  3               p & q
   7           eta   3,6       - r
   8           alpha 6         p
   9           alpha 6         q
               close 8,4
  10     br-2  3               - (p & q)
  11           eta   3,10      r
               open [r, - p, - q]

[- (p + q), - (- (p & q) -> r), p & q <-> r]
   1     - (p + q)
   2     - (- (p & q) -> r)
   3     p & q <-> r
   4     alpha 1          - p
   5     alpha 1          - q
   6     alpha 2          - (p & q)
   7     alpha 2          - r
   8     eta   3,6        - r
         open [- p, - q, - r]

[- (p + q), - (- (p & q) -> r), - (p & q <-> r)]
   1     - (p + q)
   2     - (- (p & q) -> r)
   3     - (p & q <-> r)
   4     alpha 1          - p
   5     alpha 1          - q
   6     alpha 2          - (p & q)
   7     alpha 2          - r
   8     eta   3,6        r
         close 8,7
```

accept, accept, accept

accept, accept, reject

accept, reject, reject

reject, accept, reject

reject, reject, accept

/* ** (b) */

In Predicate Logic

1	$\forall x.martyr(x) \rightarrow phlegmatic(x)$	premise
2	$\exists x.cougher(x) \wedge martyr(x)$	premise
3	$\neg(\exists x.cougher(x) \wedge phlegmatic(x))$	neg-conc
4	$cougher(c) \wedge martyr(c)$	$\exists$, 2
5	$cougher(c)$	α, 4
6	$martyr(c)$	α, 4
7	$martyr(c) \rightarrow phlegmatic(c)$	$\forall$, 1
8	$\neg(cougher(c) \wedge phlegmatic(c))$	$\forall$, 3
9	$phlegmatic(c)$	*beta*, 6, 7
10	$\neg phlegmatic(c)$	*beta*, 5, 8
	close (9,10)	

/* ** (c) */

In S5: we just label the world with integers, we can move to any world that is available on the branch

(i)

1	[1]:	$\neg(\Box p \rightarrow \Diamond p)$	$\neg conc$
2	[1]:	$\Box p$	$\alpha, 1$
3	[1]:	$\neg\Diamond p$	$\alpha, 1$
4	[1];	p	$\Box$, 2
5	[1];	$\neg p$	$\Box$, 3
		close (4,5)	

(ii)

1	[1]:	$\neg(p \rightarrow \Box\Diamond p)$	$\neg conc$
2	[1]:	p	$\alpha, 1$
3	[1]:	$\neg(\Box\Diamond p)$	$\alpha, 1$
4	[2];	$\neg(\Diamond p)$	$\Diamond$, 3
5	[1]:	$\neg p$	$\Box$, 4
		close (2,5)	

(iii)

1	[1]:	$\neg(\Diamond p \rightarrow \Box\Diamond p)$	$\neg conc$
2	[1]:	$\Diamond p$	$\alpha, 1$
3	[1]:	$\neg(\Box\Diamond p)$	$\alpha, 1$
4	[2];	p	$\Diamond$, 2
5	[3]:	$\neg\Diamond p$	$\Diamond$, 3
6	[2]:	$\neg p$	$\Box$, 5
		close (4,6)	

5. This question is about search algorithms for 2-player games, and the Simplified Event Calculus (EC).

a) Explain the difference between the MiniMax, MiniMax to Fixed Ply, and AlphaBeta search algorithms for 2-player games.

[6]

b) Consider the graph shown in Figure 5.3. Suppose that the Minimax algorithm to 3-ply would produce the evaluation of the leaf nodes as shown.

Using the AlphaBeta algorithm, identify which branches would be pruned, and explain why.

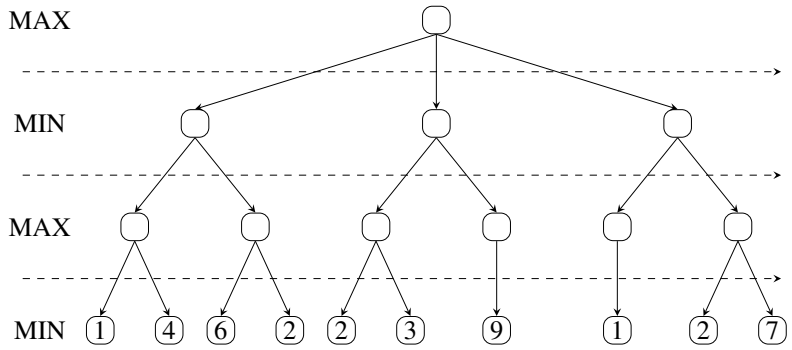


Figure 5.1 Hypothetical search space using MiniMax to fixed ply

[4]

c) The Blocks World scenario imagines a set of movable coloured blocks situated on a table, as illustrated in Figure 5.2.

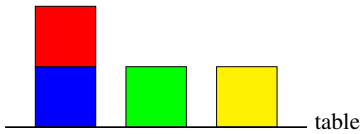


Figure 5.2 Blocks World: Initial State

There is a robot which can perform two actions: it can *stack* a block by taking any single block that is on the table and putting it on any other block, and it can *unstack* any block that is on another block. The goal is to build stacks of blocks called *towers* in a specified arrangement of colours.

Making any reasonable assumptions, specify the initial state of the Blocks World using `initially/1` facts about an identified multi-valued fluent `isOn`, whose value range will be `table` if the block is on the table, or otherwise another block.

Define `initiates/3` axioms of the Simplified EC for reasoning about the effects of *stack* and *unstack* actions. The actions should be valid (i.e., cannot unstack a block that is on the table, can only stack a block that is on the table).

Write a Prolog predicate that, given a block `B` and a time `T`, succeeds if the block `B` is the top block of a tower greater than height 1, and computes the tower itself.

[10]

(a)

Minimax: exhaustive search, label levels according to whose turn, assign leaf nodes 1 or 0 for win or lose, propagate values up tree (Min to Max, Max parent gets maximal value of Min children; Max to Min: Min parent gets minimal value of Max children

Minimax to fix ply: exhaustive search only to fix ply, apply heuristic function at leaf nodes, propagate values up tree according to same rules as Minimax

AlphaBeta, depth first search, use heuristic function to assign value, propagate value up tree, use those to decide whether or not to prune branch

(b)

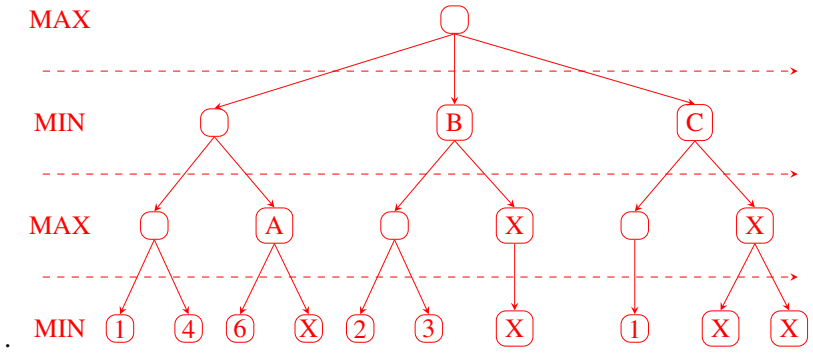


Figure 5.3 Hypothetical search space using MiniMax to fixed ply

At Node A, Alpha=4, left child offers value 6. If value from right branch < 6, it will lose going Min-to-Max. But if it is > 6, it will lose to 4 going Max to Min.

At Node B, Beta=4, left child offers value 3. If value from right branch > 3, it will lose going Max-to-Min. But if it is < 3, it will lose to 4 going Min to Max

Same reasoning applies to C

```
        /*
** (c)
*/
initially (isOn(red)=blue, 0 ).
initially (isOn(blue)=table, 0 ).
initially (isOn(green)=table, 0 ).
initially (isOn(yellow)=table, 0 ).

initiates( stack(X,Y), isOn(X)=Y, T ) :-
        holdsAt( isOn(X)=table, T ),            %on the table
        \+ holdsAt( isOn(_)=Y, T ).             %nothing on it

initiates( unstack(X), isOn(X)=table, T ) :-
        \+ holdsAt( isOn(X)=table, T ).         %can't unstack on the table
        \+ holdsAt( isOn(_)=X, T ).             %can't take from middle (not playing Jenga)

makeTower( B, Tower, T ) :-
        \+ holdsAt( isOn(_)=B, T ),             %got to be top block
        \+ holdsAt( isOn(B)=table, T ),         %got to be greater than 1
        goonthen( B, [], Tower, T ).            %use accumulator, build tower in reverse

goonthen( B, Tower, [B|Tower], T ) :-           %stick B on bottom of tower, if...
        holdsAt( isOn(B)=table, T ), !.         %B is on the table

goonthen( B, SoFar, Tower, T ) :-
        holdsAt( isOn(B)=Next, T ),             %what block is B on (i.e. Next)
        goonthen( Next, [B|SoFar], Tower, T ).  %what sub-tower is Next top of
```